

Relatório Final do Projecto VUTIVI

Table of Contents

Relatório Final do Projecto VUTIVI	1
Capa	1
1. Introdução	2
2. Contextualização do Problema	3
3. Objectivos do Projecto	5
4. Metodologia de Desenvolvimento	5
5. Descrição da Aplicação VUTIVI	6
6. Histórico de Versões	7
7. Funcionalidades Implementadas	8
8. Arquitectura do Sistema	11
9. Modelos e Base de Dados	12
10. Rotas Principais	13
11. Diagramas Explicativos	14
12. Segurança e Validações	14
13. Testes Realizados	15
14. Resultados Alcançados	15
15. Funcionalidades Pendentes (Backlog)	16
16. Conclusão	17
17. Recomendações Futuras	17
18. Nota Para Conversão em DOCX	17

Relatório Final do Projecto VUTIVI

Capa

Projecto

VUTIVI

Tema

Plataforma de gestão de biblioteca física e digital

Tecnologia principal

Laravel (PHP), PostgreSQL, Blade, Tailwind CSS, Vite

Idioma	Português de Moçambique
Natureza do documento	Relatório técnico e expositivo do sistema desenvolvido
Versão do documento	1.0
Data	2026-06-21

Estudantes:

Nome	Função no Projecto
Jose Zeferino Chauque Jr	Desenvolvimento e documentação
Bonifacio Simbine	Desenvolvimento e documentação

1. Introdução

O presente relatório descreve o projecto VUTIVI, uma aplicação web desenvolvida para apoiar a gestão de bibliotecas físicas e digitais. A plataforma foi concebida para organizar recursos, controlar empréstimos, gerir acessos digitais, acompanhar prazos e melhorar a experiência de utilizadores que procuram, partilham e utilizam materiais de estudo.

O nome VUTIVI vem do xitsonga, em que significa “conhecimento”. A escolha do nome não é meramente estética: reflecte o propósito central do projecto, que é tratar o conhecimento como um bem que circula, em vez de um conjunto de registos estáticos e isolados numa estante ou numa pasta de ficheiros. A aplicação procura transformar a biblioteca num ambiente mais acessível, rastreável e colaborativo, onde cada recurso possui informação clara, estado actualizado e ligação directa ao seu dono e aos seus utilizadores.

Historicamente, a biblioteca desempenha um papel central na democratização do acesso ao saber, funcionando como ponto de encontro entre quem produz, partilha e procura informação. Contudo, a forma de gerir esse acervo evoluiu pouco em muitos contextos: catálogos em papel, cadernos de registo de empréstimo, controlo manual de prazos e comunicação informal entre bibliotecário e utilizador continuam a ser a norma em diversas instituições de pequena e média dimensão. A VUTIVI nasce da constatação de que esse modelo, apesar de funcional em escalas reduzidas, deixa de responder com eficiência quando o volume de recursos, utilizadores e empréstimos simultâneos aumenta.

O projecto foi pensado para ir além de um simples sistema de catalogação. Ele cobre o ciclo de vida completo de um recurso — desde o registo inicial, passando pela publicação no catálogo, pela pesquisa e descoberta por outros utilizadores, pelo empréstimo (físico ou acesso digital), até à devolução, eventual penalização por atraso e avaliação pela comunidade. Esta visão de ciclo completo é o que distingue a VUTIVI de uma simples lista

de inventário: o sistema acompanha o recurso e o utilizador em todas as fases da relação entre ambos.

A plataforma foi desenvolvida com tecnologias web modernas — Laravel no backend, Blade e Tailwind CSS no frontend, e PostgreSQL como base de dados — escolhidas pela maturidade, robustez e disponibilidade de documentação, factores relevantes num projecto académico com prazo definido e equipa reduzida. A arquitectura adoptada privilegia a separação de responsabilidades, a validação rigorosa de dados e a cobertura por testes automatizados, de modo a garantir que o sistema permaneça compreensível e extensível mesmo após o término do projecto curricular que o originou.

Este documento consolida a documentação técnica produzida durante o desenvolvimento (docs/funcionalidades-do-sistema.md, docs/vutivi-documentacao-tecnica.md, docs/relatorio-vutivi.md, README.md e docs/notes.md) com o levantamento directo das funcionalidades efectivamente implementadas no código-fonte e com o histórico de versões registado no controlo de versões (Git). O objectivo é oferecer, num único relatório, uma visão completa e verificável do que foi efectivamente construído, evitando a dispersão de informação entre múltiplos ficheiros de apoio que foram sendo criados ao longo das diferentes fases do desenvolvimento.

A estrutura do relatório segue uma progressão lógica: começa por enquadrar o problema e os objectivos (secções 2 e 3), descreve a metodologia e a aplicação (secções 4 e 5), apresenta o histórico de versões e as funcionalidades implementadas (secções 6 e 7), detalha a arquitectura, os modelos de dados e as rotas (secções 8 a 10), explica os fluxos de utilização através de diagramas (secção 11), descreve as medidas de segurança e os testes realizados (secções 12 e 13), e termina com os resultados, o trabalho pendente, as conclusões e as recomendações futuras (secções 14 a 17).

2. Contextualização do Problema

Em muitas instituições de ensino, bibliotecas comunitárias e centros de formação — particularmente em contextos como o moçambicano, onde a infra-estrutura tecnológica das bibliotecas é frequentemente limitada — a gestão de livros, documentos, apostilas, vídeos e ficheiros digitais ainda depende de processos manuais ou dispersos. É comum encontrar o registo de empréstimos feito em cadernos físicos, fichas de cartolina ou, na melhor das hipóteses, folhas de cálculo isoladas que não comunicam entre si nem com o restante acervo. Essa realidade cria dificuldades na localização de recursos, no controlo de empréstimos, na identificação de materiais indisponíveis e no acompanhamento de devoluções.

As consequências práticas desta dispersão são sentidas tanto pelos utilizadores como pelos responsáveis pela gestão da biblioteca. Do lado do utilizador, a procura por um recurso específico pode exigir deslocação física, consulta a múltiplos cadernos ou dependência da memória do bibliotecário, sem qualquer garantia de que o material esteja

realmente disponível no momento da consulta. Do lado da gestão, a ausência de um sistema centralizado dificulta saber, em tempo real, quantos exemplares de um recurso existem, quem os tem em sua posse, quais empréstimos estão em atraso e que multas ainda não foram regularizadas. Sem visibilidade sobre estes indicadores, decisões como renovar o acervo, cobrar atrasos ou identificar utilizadores reincidentes tornam-se lentas e sujeitas a erro humano.

Para além da dimensão puramente operacional, existe também uma dimensão de experiência e participação: bibliotecas tradicionais raramente oferecem aos seus utilizadores mecanismos de avaliação de recursos, listas de leitura pessoais ou recomendações baseadas em histórico, elementos hoje comuns em plataformas digitais de conteúdo e que aumentam o envolvimento da comunidade com o acervo disponível. A ausência destes mecanismos reduz o potencial da biblioteca como espaço de descoberta e partilha de conhecimento, limitando-a a uma função puramente transaccional de “pedir e devolver”.

Entre os principais problemas identificados destacam-se:

- Falta de um catálogo digital centralizado, com recursos físicos e digitais tratados de forma incoerente entre si.
- Dificuldade em controlar prazos de empréstimo e em identificar atrasos de forma proactiva.
- Ausência de notificações para devoluções e extensões, deixando a comunicação dependente de contacto directo e informal.
- Pouca rastreabilidade sobre quem possui determinado recurso em cada momento, o que dificulta a responsabilização e a recuperação de materiais em falta.
- Falta de distinção clara entre as regras aplicáveis a recursos físicos (localização, condição, devolução obrigatória) e digitais (ficheiro, tipo e janela de acesso).
- Processo manual para gerir pedidos, aprovações, devoluções e multas por atraso, sujeito a inconsistências e à indisponibilidade do responsável.
- Ausência de ferramentas de gestão dedicadas para bibliotecários, que precisam de uma visão agregada e operacional do estado de todos os empréstimos.
- Pouca ou nenhuma participação dos utilizadores na vida do acervo, sem espaço para avaliações, listas de leitura ou histórico pessoal de pesquisa.
- Falta de relatórios objectivos que permitam fundamentar decisões de gestão com dados reais em vez de percepções.

A VUTIVI surge como resposta directa a este cenário, oferecendo uma solução integrada para organização, pesquisa, empréstimo, fiscalização, comunicação e acompanhamento de recursos, substituindo processos dispersos e manuais por um fluxo único, auditável e acessível a partir de qualquer dispositivo com acesso à internet.

3. Objectivos do Projecto

3.1 Objectivo Geral

Desenvolver uma plataforma web para gestão de biblioteca física e digital, permitindo o registo, pesquisa, empréstimo, devolução, notificação e acompanhamento de recursos de forma organizada e segura.

3.2 Objectivos Específicos

#	Objectivo
1	Permitir o cadastro e autenticação de utilizadores com diferentes níveis de acesso
2	Permitir o registo de recursos físicos e digitais
3	Disponibilizar uma biblioteca com pesquisa de texto completo, filtros e paginação
4	Controlar pedidos de empréstimo, devoluções e histórico
5	Permitir pedidos de extensão de prazo
6	Notificar donos de recursos e utilizadores sobre decisões relevantes
7	Garantir sincronização entre estado do empréstimo e disponibilidade do recurso
8	Aplicar multas automáticas por atraso na devolução
9	Disponibilizar um painel de gestão dedicado a bibliotecários
10	Permitir avaliações, listas de leitura e relatórios exportáveis
11	Validar dados de entrada e proteger rotas sensíveis
12	Criar uma interface responsiva, moderna e simples de utilizar

4. Metodologia de Desenvolvimento

O desenvolvimento seguiu uma abordagem incremental, com separação clara entre regras de negócio, interface, persistência e validação. A aplicação foi estruturada com

base no padrão MVC do Laravel, complementado por classes de acção (Actions) para isolar operações específicas.

4.1 Princípios Aplicados

Princípio	Aplicação Prática
Validação de entrada em FormRequest	store e update nunca validam dados directamente
Persistência em Actions	Cada operação crítica tem uma classe dedicada (ex.: CreateReservation)
Controllers apenas orquestram	Recebem requisição, chamam request/action, devolvem resposta HTTP
Regras de negócio críticas isoladas	Excepções e validações dedicadas (ex.: InvalidCredentialsException)
Funções pequenas e de responsabilidade única	Cada método faz uma única coisa

4.2 Práticas Complementares

- Modelação de entidades com migrations e relacionamentos Eloquent.
- Implementação de views Blade com componentes reutilizáveis (navbar, sidebar, breadcrumbs, toasts).
- Criação de testes automatizados para proteger comportamentos essenciais.
- Uso de feedback visual por toasts, badges, estados e notificações.
- Controlo de acesso baseado em papéis (utilizador, bibliotecário, administrador).

5. Descrição da Aplicação VUTIVI

A VUTIVI é uma plataforma híbrida de biblioteca. Permite gerir tanto recursos físicos, como livros e materiais impressos, quanto recursos digitais, como PDFs, vídeos, áudios, documentos e apresentações.

O sistema organiza cada recurso com dados comuns — título, descrição, dono, estado, capa, slug e tipo — e adiciona informações específicas conforme o recurso seja físico ou digital.

Tipo de Recurso	Dados Específicos
Físico	Localização, condição, quantidade disponível, prazo máximo de empréstimo, aprovação, número máximo de extensões
Digital	Caminho do ficheiro, tipo de acesso, dias de acesso, modo de visualização ou download

6. Histórico de Versões

A tabela seguinte regista a evolução da aplicação ao longo do desenvolvimento, organizada por marcos funcionais a partir do histórico de commits do repositório Git.

Versão	Data	Commit(s)	Marco / Descrição
v0.1.0	2026-04-28	73fa289	Estrutura inicial da aplicação Laravel: rotas, testes e configuração de base.
v0.2.0	2026-05-09	12b400b	Adição de feedback visual (mensagens/toasts) nas views de recursos.
v0.3.0	2026-05-14	be43c0b, 5d76cd3, 819eb8d	Rebranding para “Vutivi Library”: UI responsiva, suporte a temas, cadastro e cards dinâmicos de recursos, navegação autenticada.
v0.4.0	2026-05-20	153c342, c7f2e75, e89620d, 42e3398, b69edb5, 8839e80	Termos e condições com registo de aceitação; melhoria do fluxo de empréstimo, UX de reservas e lógica de disponibilidade; notificações; documentação do sistema; deck de apresentação interactivo.
v0.4.1	2026-05-21	c325c01	Correcção de conflitos de migrations e estabilização do schema de reservas.
v0.5.0	2026-05-25	ce5fe38, 210a1e4	Responsividade

Versão	Data	Commit(s)	Marco / Descrição
			mobile (grids adaptáveis, cards em coluna única), optimização de fontes (carregamento local via @font-face) e melhorias visuais nos cards.
v0.6.0	2026-05-28	aec1936, 6928ba7, 32741be, e610279, f8eceb0, 9abc89a	Segurança e auditoria na fila de espera de reservas; notificação ao dono quando o recurso é devolvido; dashboard, avaliações (reviews), listas de leitura, pesquisa de texto completo, multas por atraso e relatórios; painel de bibliotecário; refactor de estrutura HTML; limite de foto de perfil a 2 MB.

Estado actual: v0.6.0 — versão estável com os módulos principais de catálogo, empréstimos, multas, avaliações, listas de leitura, relatórios e gestão por bibliotecário implementados.

7. Funcionalidades Implementadas

A tabela seguinte resume todas as funcionalidades confirmadas no código-fonte, com o respectivo estado e os componentes responsáveis.

Funcionalidade	Estado	Controller / Action Principal	Descrição Resumida
Autenticação e Conta	Completo	Auth\LoginController, AuthenticateUser,	Login, logout e protecção de rotas privadas por

Funcionalidade	Estado	Controller / Action Principal	Descrição Resumida
		LogoutUser	middleware.
Gestão de Utilizadores	Completo	UserController, DeleteUser	Cadastro, edição de perfil, upload de foto (limite 2 MB) e exclusão de conta.
Catálogo e Biblioteca	Completo	ResourceController	Listagem de recursos disponíveis com pesquisa, filtros e paginação.
Pesquisa de Texto Completo	Completo	ResourceController::library()	Pesquisa via tsvector/tsquery (PostgreSQL) com fallback LIKE e histórico de buscas.
Recursos Físicos	Completo	PhysicalResourceController, CreatePhysicalResource, UpdatePhysicalResource	CRUD de recursos físicos com localização, condição e prazo.
Recursos Digitais	Completo	DigitalResourceController, CreateDigitalResource, UpdateDigitalResource	CRUD de recursos digitais com ficheiro, tipo e janela de acesso.
Empréstimos (Reservas)	Completo	ReservationController, CreateReservation, UpdateReservation, ReturnReservation, ValidateReservationAgainstResource, SyncResourceAvailability	Criação, actualização, devolução e exclusão de reservas com validação de negócio.
Extensão de Prazo	Completo	ReservationController (request-extension, approve-extension, deny-extension)	Pedido de extensão pelo cliente e decisão pelo dono do recurso.
Multas por Atraso	Completo	FineController,	Emissão automática

Funcionalidade	Estado	Controller / Action Principal	Descrição Resumida
(Fines)		IssueFine	de multa por dia de atraso e marcação de pagamento.
Avaliações (Reviews)	Completo	ResourceReviewController	Avaliação de 1 a 5 estrelas com comentário, restrita a quem devolveu o recurso.
Listas de Leitura	Completo	ReadingListController	Listas personalizadas, com três listas padrão por utilizador.
Relatórios (Reports)	Completo	ReportController	Exportação em CSV de empréstimos activos, atrasados, histórico e multas.
Painel de Bibliotecário	Completo	LibrarianController, LibrarianMiddleware	Gestão de pedidos, levantamentos, devoluções e multas pela equipa da biblioteca.
Controlo de Acesso por Papéis	Completo	User::isAdmin(), isLibrarian(), canManageLoans()	Três níveis: utilizador, bibliotecário e administrador.
Notificações	Completo	Vários controllers/actions	Alertas de prazo, extensão, aprovação, recusa e devolução.
Favoritos	Completo	ResourceController (JSON)	Marcação optimista de favoritos com confirmação assíncrona.
Acesso Digital Avançado	Incompleto (esqueleto)	DigitalAccess, DigitalAccessController	Estrutura inicial criada; faltam regras de negócio, rotas, views e testes.

8. Arquitectura do Sistema

A aplicação foi construída com Laravel, utilizando o padrão MVC e Blade para renderização das páginas, Tailwind CSS para estilo e Vite para build dos assets.

8.1 Camadas Principais

Camada	Responsabilidade	Exemplos
Models	Entidades persistentes e relacionamentos	User, Resource, PhysicalResource, DigitalResource, Reservation, ResourceReview, ReadingList, Fine
Controlllers	Coordenação de pedidos HTTP e respostas	ResourceController, ReservationController, LibrarianController, ReportController
Requests	Validação e normalização de dados	StoreUserRequest, StoreReservationRequest, UpdatePhysicalResourceRequest
Actions	Regras de negócio reutilizáveis	CreateReservation, SyncResourceAvailability, IssueFine, DeleteUser
Views Blade	Apresentação ao utilizador	navbar, sidebar, breadcrumbs, toasts, resource card
Migrations	Estrutura da base de dados	users, resources, reservations, fines, resource_reviews
Middleware	Protecção de rotas	auth, guest, LibrarianMiddleware
Tests	Verificação de comportamento esperado	tests/Feature/*

8.2 Separação de Responsabilidades

A separação de responsabilidades torna o sistema mais legível, testável e extensível. Por exemplo, a devolução de uma reserva é tratada pela action ReturnReservation, a emissão de multa pela action IssueFine, enquanto o controller apenas coordena a chamada e a resposta ao utilizador.

9. Modelos e Base de Dados

Model	Tabela	Dados Principais
User	users	Nome, username, slug, email, foto de perfil, senha, papel (role)
Resource	resources	Título, slug, descrição, capa, tipo, estado, quantidade, dono, vector de pesquisa
PhysicalResource	physical_resources	Localização, condição, prazo máximo, aprovação, máximo de extensões
DigitalResource	digital_resources	Caminho do ficheiro, tipo de acesso, dias de acesso
Reservation	reservations	Utilizador, recurso, datas, estado, aprovação, devolução, extensão
Fine	fines	Dias de atraso, taxa diária, total, data de pagamento
ResourceReview	resource_reviews	Classificação (1–5), comentário, utilizador, recurso
ReadingList / ReadingListItem	reading_lists, reading_list_items	Nome da lista, recursos associados
SearchHistory	search_history	Termos pesquisados, utilizador, data
TermAndCondition / TermAcceptance	terms_and_conditions, term_acceptances	Termos aplicáveis e aceitação pelo utilizador
Pivot Favoritos	resource_user_favorites	Relação muitos-para-muitos entre utilizadores e recursos

9.1 Relacionamentos Principais

Relação	Tipo
User → Resource	hasMany
User → Reservation	hasMany
User ↔ Resource (favoritos)	belongsToMany
Resource → User	belongsTo
Resource → PhysicalResource	hasOne
Resource → DigitalResource	hasOne

Relação	Tipo
Resource → Reservation	hasMany
Reservation → Resource	belongsTo
Reservation → User	belongsTo

10. Rotas Principais

Módulo	Rotas
Autenticação	GET/POST /login, POST /logout
Utilizadores	GET/POST /register, GET /users/{user}/edit, PUT/DELETE /users/{user}
Catálogo	GET /resources, GET /resources/{resource}, DELETE /resources/{resource}
Biblioteca pública	/library, /sobre, /recurso/{slug}
Recursos Físicos	GET/POST /physical-resources, GET/PUT/DELETE /physical-resources/{id}
Recursos Digitais	GET/POST /digital-resources, GET/PUT/DELETE /digital-resources/{id}
Reservas	GET/POST /reservations, GET/PUT/DELETE /reservations/{id}, PATCH /reservations/{id}/return
Extensão de Prazo	/reservations/{id}/request-extension, /approve-extension, /deny-extension
Avaliações	POST/DELETE /resources/{resource}/reviews
Listas de Leitura	/reading-lists, /reading-lists/{list}/items
Multas	GET /fines
Relatórios	/reports, /reports/active-loans, /reports/overdue-loans, /reports/loan-history, /reports/fines
Bibliotecário	/librarian/dashboard (middleware librarian)

11. Diagramas Explicativos

11.1 Casos de Uso

Actores principais: Utilizador, Dono do recurso, Bibliotecário, Administrador, Sistema.

Caso de Uso	Actor
Pesquisar recurso (texto completo)	Utilizador
Consultar detalhes	Utilizador
Solicitar empréstimo	Utilizador
Devolver recurso	Utilizador
Pedir extensão	Utilizador
Aprovar ou recusar extensão	Dono do recurso
Avaliar recurso devolvido	Utilizador
Gerir listas de leitura	Utilizador
Pagar/consultar multas	Utilizador
Gerir recurso	Dono do recurso
Gerir empréstimos da biblioteca	Bibliotecário
Exportar relatórios	Utilizador / Bibliotecário
Receber notificação	Sistema → Utilizador

11.2 Relacionamento de Dados

O utilizador possui muitos recursos, muitas reservas, muitas avaliações e muitas listas de leitura. O recurso pertence a um utilizador, pode ter um detalhe físico ou digital, e pode possuir muitas reservas e avaliações. As reservas ligam utilizadores a recursos durante o ciclo de empréstimo e podem originar multas.

11.3 Classes Principais

As classes principais do domínio são User, Resource, PhysicalResource, DigitalResource, Reservation, Fine, ResourceReview, ReadingList, TermAndCondition e TermAcceptance. Cada uma possui responsabilidade específica e relacionamentos que reflectem a lógica da biblioteca.

12. Segurança e Validações

Mecanismo	Aplicação
Middleware de autenticação	Protege rotas privadas (auth)
Middleware de bibliotecário	Restringe /librarian/dashboard (LibrarianMiddleware)

Mecanismo	Aplicação
Form Requests	Validação de todos os formulários de entrada
Protecção CSRF	Formulários e chamadas assíncronas (fetch)
Autorização por dono	Apenas o dono edita ou decide sobre o seu recurso
Bloqueio de auto-empréstimo	Impede emprestar o próprio recurso
Validação de disponibilidade	Antes de criar qualquer empréstimo
Controlo de acesso por papéis	<code>isAdmin()</code> , <code>isLibrarian()</code> , <code>canManageLoans()</code>
Hash de senhas	Automático via Laravel
Validação de uploads	Tipo e tamanho (ex.: foto de perfil ≤ 2 MB)

13. Testes Realizados

Ficheiro de Teste	Área Coberta
<code>tests/Feature/Auth/LoginTest.php</code>	Autenticação
<code>tests/Feature/Users/UserControllerTest.php</code>	Cadastro e gestão de utilizadores
<code>tests/Feature/Resources/DigitalResourceCrudTest.php</code>	CRUD de recursos digitais
<code>tests/Feature/Resources/PhysicalResourceCrudTest.php</code>	CRUD de recursos físicos
<code>tests/Feature/Resources/ResourceCrudTest.php</code>	Catálogo partilhado de recursos
<code>tests/Feature/Resources/ResourceSchemaTest.php</code>	Schema e integridade dos recursos
<code>tests/Feature/Reservations/ReservationCrudTest.php</code>	Reservas, devolução e disponibilidade

No estado da suíte documentada, foram registados **16 testes aprovados** e **118 asserções**, indicando que os principais contratos da aplicação estão protegidos.

14. Resultados Alcançados

Resultado	Descrição
Catálogo centralizado	Recursos físicos e digitais com pesquisa de texto completo
Fluxo de empréstimo completo	Criação, aprovação, devolução, histórico e

Resultado	Descrição
Extensões de prazo	multas automáticas Pedido, aprovação e recusa pelo dono do recurso
Gestão operacional	Painel dedicado para bibliotecários
Engajamento do utilizador	Avaliações, listas de leitura e favoritos
Transparência de dados	Relatórios exportáveis em CSV
Comunicação	Notificações para utilizadores e donos de recursos
Consistência de dados	Sincronização entre estado da reserva e disponibilidade do recurso
Controlo de acesso	Três níveis de papel: utilizador, bibliotecário, administrador
Experiência visual	Interface responsiva com cards, filtros, navbar, sidebar e toasts

15. Funcionalidades Pendentes (Backlog)

Conforme registado em docs/notes.md e confirmado no estado actual do código:

#	Pendência	Estado
1	Notificar o dono do recurso sobre pedido de extensão, com aprovação/recusa e notificação ao cliente	Parcialmente implementado
2	Ajustar o fluxo de logout para direccionar à página inicial	Pendente
3	Corrigir o problema em que, após devolução, o utilizador não consegue voltar a ter acesso ao mesmo recurso	Pendente
4	Completar o módulo de Acesso Digital (DigitalAccess/DigitalAccessController)	Pendente (esqueleto inicial)

16. Conclusão

A VUTIVI demonstra como uma biblioteca pode ser modernizada através de uma aplicação web organizada, responsiva e orientada a processos reais. A plataforma reduz a dispersão de informação, melhora o controlo de empréstimos e oferece uma experiência mais clara para utilizadores, donos de recursos e bibliotecários.

O sistema não se limita ao cadastro de materiais. Acompanha o ciclo completo do recurso, desde a publicação até à devolução, incluindo notificações, favoritos, avaliações, listas de leitura, multas, relatórios e gestão operacional por papéis de utilizador.

17. Recomendações Futuras

#	Recomendação
1	Implementar notificações persistentes em base de dados
2	Expandir o dashboard administrativo com mais métricas
3	Criar API REST para integração com outras plataformas
4	Implementar broadcasting em tempo real
5	Adicionar auditoria de acções sensíveis
6	Criar recomendações de recursos com base em favoritos, avaliações e histórico
7	Melhorar permissões com policies específicas do Laravel
8	Concluir o módulo de Acesso Digital
9	Resolver os itens pendentes do backlog (logout, reaccesso após devolução, notificação de extensão)

18. Nota Para Conversão em DOCX

Este relatório está estruturado em Markdown para conversão simples em DOCX. Pode ser convertido com uma das seguintes opções:

1. Abrir este ficheiro num editor compatível, como Microsoft Word ou LibreOffice, e guardar como .docx.
2. Usar Pandoc:

```
pandoc docs/relatorio-final-vutivi.md -o docs/relatorio-final-vutivi.docx
```

O conteúdo está organizado com títulos, subtítulos, tabelas e secções formais para manter uma estrutura adequada em formato de relatório académico.